# Mercredi 18/11 — Soutien & autonomie

Kevin Bardoux

mercredi 18 novembre 2026

## Table des matières

Matin — Soutien · premier clone de dwwm-soutien · fin de « Corriger et republier » ..... 2
Soutien ..... 2
Le soutien de ce matin ..... 2
Déroulé ..... 2
Après-midi — Autonomie : parcours d'autonomie · échéance du composant au portfolio ..... 3
Consignes ..... 3
Parcours d'autonomie de la semaine ..... 4
Ligne 1 : cloner `dwwm-soutien` ..... 5
Ligne 2 : finir « Corriger et republier » ..... 5
Ligne 3 : le composant au portfolio ..... 5
Ligne 4 : les exercices de palier 2 de lundi et de mardi ..... 6
Ligne 5 : mission de documentation — la position d'un musée ..... 6
Ligne 6 : les exercices de palier 3 de lundi et de mardi ..... 7
Ligne 7 : pour aller plus loin — les fonctions qui fabriquent des fonctions ..... 7
Sources ..... 9

> **Note**
>
> **Soutien prévu pour vous** : le matin avec le formateur, puis l'après-midi (12 h 45 – 17 h) en autonomie. **Pas de soutien prévu** : en autonomie toute la journée, seul ou en binôme (parcours d'autonomie, puis l'activité de l'après-midi). **À déposer ce soir** : dans `semaine-07/preuves.md` de votre dépôt `dwwm-tp`, l'adresse HTTPS de votre composant publié et le lien de votre dernier commit, puis `git push`.

# Matin — Soutien · premier clone de dwwm-soutien · fin de « Corriger et republier »

## Soutien

Si le formateur vous a indiqué des exercices, passez la matinée avec lui : la page Soutien de la semaine explique comment les récupérer.

Sans soutien prévu, vous êtes en autonomie toute la journée : commencez directement le parcours d'autonomie.

## Le soutien de ce matin

- **Dépôt** : dwwm-soutien, cloné pour la première fois.
- **Objectif** : vos exercices faits, et la page perso corrigée.
- **Pourquoi** : retravailler ce qui vous bloque.

## Déroulé

1. **Rappel** (5 min, à 9 h 05) : trois questions courtes.
2. **Premier clone** (10 min) : `dwwm-soutien`, ensemble, en 4 étapes.
3. **Pratique accompagnée** : vos exercices, par groupes de 3 à 5.
4. **Point d'étape** (après la pause) : une difficulté commune.
5. **« Corriger et republier »** : pour ceux qui ne l'ont pas fini.

**Rappel.** 1. Que renvoie `find` quand aucun élément ne convient ? 2. Où se met `display: flex` ? 3. Que fait `git clone` ?

**Premier clone, en 4 étapes** (le détail et les captures sont sur la page Soutien de la semaine) :

1. Dans le terminal de VS Code, allez dans votre dossier de formation : `cd ~/dwwm`.
2. Clonez le dépôt : `git clone https://github.com/dwwm2627/dwwm-soutien.git`.
3. Ouvrez le fichier de la semaine dans le dossier de votre numéro : `E07/s07.md`, par exemple.
4. Copiez votre premier exercice dans `~/dwwm/dwwm-tp/semaine-07/soutien/`, puis ouvrez la copie dans VS Code.

**Vous devez voir** : un dossier `~/dwwm/dwwm-soutien`, à côté de `dwwm-tp`, et la liste de vos exercices de la semaine.

**« Corriger et republier » (semaine 6).** Ce qui n'était pas fini vendredi 13/11 se termine ce matin : reprenez la page Vendredi 13/11, « Corriger et republier ». **Réussi si** : en ligne, Lighthouse donne au moins 90 en accessibilité, tout se fait au clavier, et le contraste du texte est d'au moins 4,5:1.

## Après-midi — Autonomie : parcours d'autonomie · échéance du composant au portfolio

- **Objectif** : votre composant publié, et 2 exercices de palier 2 validés.
- **Preuve** : l'adresse du composant, le lien du commit.
- **Échéance du composant** : aujourd'hui, 17 h.

### Consignes

1. **Le composant au portfolio** (1 h 30 au plus) : finissez-le et publiez-le, comme votre page perso jeudi dernier, dans un **dépôt public à part** (les consignes sont dans la ligne 10 du parcours de la semaine 6). Ce qui est public est visible par tous : aucune donnée personnelle dedans. **Réussi si** : l'adresse HTTPS s'ouvre, et le composant colle à la maquette à 375 px et en bureau.

2. **Deux exercices de palier 2** de lundi ou de mardi, au choix (1 h). **Réussi si** : `node --test nom.test.js` affiche `i fail 0` pour chacun, et un commit par exercice.
3. **La preuve** : en ligne de commande, créez `semaine-07/preuves.md` ; écrivez-y l'adresse HTTPS du composant et le lien de votre dernier commit (sur GitHub, cliquez sur le commit, puis copiez l'adresse de la page). Puis `git add`, `git commit -m "Ajoute les preuves du mercredi"`, `git push`.
4. **La suite** : le parcours d'autonomie, ci-dessous, là où vous en êtes.

> Si vous êtes bloqué
>
> - La publication ne s'affiche pas : attendez deux minutes, puis rechargez ; vérifiez *Settings* > *Pages* (branche `main`, dossier `/(root)`), comme dans la fiche de révision de la semaine 6.
> - Un test échoue : la méthode « Si l'on bloque », plus bas.
> - Demandez à CS50.ai d'expliquer un message d'erreur, pas de le corriger.

## Parcours d'autonomie de la semaine

Le matin pour ceux qui n'ont pas de soutien prévu ; ensuite, si l'activité est terminée, le vendredi après-midi, ou pendant le soutien pair à pair du mardi (15 h 30 – 17 h). Les exercices de soutien demandés par chacun sont dans la page Soutien de la semaine.

1. **Palier 1 — Cloner `dwwm-soutien`** (10 min) : pour ceux qui ne l'ont pas fait ce matin.
2. **Palier 2 — Finir « Corriger et republier »** (45 min) : la page perso.
3. **Palier 2 — Le composant au portfolio** (1 h 30) : publié, échéance ce soir.
4. **Palier 2 — Les exercices de lundi et de mardi** non faits.
5. **Palier 2 — Mission de documentation** (20 min) : la position d'un musée.

6. **Palier 3 — Les exercices de lundi et de mardi** non faits.
7. **Palier 3 — Pour aller plus loin** : les fonctions qui fabriquent des fonctions.

**Ligne 1 : cloner `dwwm-soutien`**
**Palier 1 : guidé.** Suivez les 4 étapes de la page Soutien de la semaine. **Réussi si** : le dossier `~/dwwm/dwwm-soutien` existe (`ls ~/dwwm` l'affiche).

> ℹ Aides
>
> - **Le cours** : Soutien de la semaine ; la fiche de révision de la semaine 4.
> - **L'IA** : CS50.ai (palier IA 1) : demandez ce que fait `git clone`, pas autre chose.
> - **MDN** : MDN ne documente pas Git : la fiche aide-mémoire Git du Drive partagé de la formation.

**Ligne 2 : finir « Corriger et republier »**
**Palier 2 : semi-guidé.** La page Vendredi 13/11, « Corriger et republier ». **Réussi si** : en ligne, Lighthouse ≥ 90 en accessibilité, tout au clavier, contraste ≥ 4,5:1.

> ℹ Aides
>
> - **Le cours** : Vendredi 13/11, « Vérifier sa page ».
> - **L'IA** : CS50.ai (palier IA 1) : demandez d'expliquer un problème signalé par Lighthouse, pas de le corriger.
> - **MDN** : Qu'est-ce que l'accessibilité ?.

**Ligne 3 : le composant au portfolio**
**Palier 2 : semi-guidé.** L'activité de l'après-midi, ci-dessus. **Réussi si** : l'adresse HTTPS s'ouvre, aux deux largeurs, sans donnée personnelle.

> ℹ Aides
>
> - **Le cours** : la ligne 10 du parcours de la semaine 6 ; la page Finition visuelle.
> - **L'IA** : CS50.ai (palier IA 1) : demandez pourquoi une mise en page déborde à 375 px, pas le code.
> - **MDN** : Boîtes flexibles ; Grilles.

**Ligne 4 : les exercices de palier 2 de lundi et de mardi**
**Palier 2 : semi-guidé.** Dans l'ordre : `l5-majuscules`, `l8-prix-ttc`, `l4-mots-longs`, `l6-prix-ttc`, `m2-majeurs`, `dc1-nettoyer-nombre`, `dc2-nettoyer-musee`. **Réussi si** : `node --test nom.test.js` affiche `ℹ fail 0` pour chacun.

> ℹ Aides
>
> - **Le cours** : Lundi après-midi, mardi matin, mardi après-midi.
> - **L'IA** : CS50.ai (palier IA 1) : demandez une explication du message d'erreur, pas le code.
> - **MDN** : Array.

**Ligne 5 : mission de documentation — la position d'un musée**
**Palier 2 : semi-guidé.** Durée : 20 min.

1. Ouvrez `f2-position.js` : `positionOf(names, name)` renvoie la **position** (l'index) d'un nom dans le tableau, ou `-1` s'il est absent.
2. Dans la page MDN de `find`, section *Voir aussi*, trouvez la méthode qui ressemble à `find` mais renvoie la position.
3. Écrivez la fonction avec cette méthode.
4. Lancez `node --test f2-position.test.js`.

**Réussi si** : `node --test f2-position.test.js` affiche `ℹ fail 0`.

ℹ Aides

- **Le cours** : Mardi matin, « filter et find ».
- **L'IA** : CS50.ai (palier IA 1) : demandez ce qu'est l'index d'un tableau, pas le nom de la méthode.
- **MDN** : Array.prototype.find(), section *Voir aussi*.

**Ligne 6 : les exercices de palier 3 de lundi et de mardi**
**Palier 3 : autonome.** `tp-my-map`, `tp-my-filter`, `tp-my-find`, `dc3-nettoyer-tout`, `o2-valeurs-par-defaut`. **Réussi si** : `node --test nom.defi.test.js` affiche `ℹ fail 0` pour chacun.

ℹ Autonome

Vous allez bloquer : c'est voulu, c'est le métier. Méthode : 1. lisez le message d'erreur en entier ; 2. cherchez-le dans la fiche « Lire un message d'erreur » ; 3. lisez la documentation ; 4. après 15 minutes bloqué sur le même point, demandez.

ℹ Aides

- **Le cours** : les pages de lundi après-midi et de mardi.
- **L'IA** : CS50.ai (palier IA 1) : demandez une explication, pas le code.
- **MDN** : Array.

**Ligne 7 : pour aller plus loin — les fonctions qui fabriquent des fonctions**
**Palier 3 : autonome.** Une fonction peut **renvoyer** une fonction, qui se souvient des variables de l'endroit où elle a été créée : c'est une *closure*. Ce n'est pas au programme de la semaine : c'est pour ceux qui ont fini tout le reste.

ℹ Ressource

**Closures** — MDN, en anglais (MDN Web Docs, s. d.) - **Pourquoi** : comprendre comment une fonction « se souvient » d'une variable. - **Quoi** : l'introduction et la partie *Lexical scoping*, avec son premier exemple. - **Durée** : 20 min · **Langue** : anglais · **Niveau** : avancé. - **Mission** : faites `c1-saluer`, puis `c2-multiplicateur` ; ensuite `c3-compteur` et `lab2-compte-bancaire`. Chaque fichier a sa consigne ; `node --test c1-saluer.defi.test.js`, et ainsi de suite.

ℹ Ressource

**Visualiseur pas à pas** — le cours, hors ligne - **Pourquoi** : voir une fonction fabriquée, et la variable qu'elle garde. - **Quoi** : les exemples « Une fonction qui fabrique une fonction », puis « Un compteur à l'état privé ». - **Durée** : 10 min · **Langue** : français · **Niveau** : avancé. - **Mission** : avant chaque **Ligne suivante**, prédisez ce qui va changer. Notez dans un commentaire de `c3-compteur.js` pourquoi deux compteurs sont indépendants.

**Réussi si** : `node --test c1-saluer.defi.test.js` affiche `ℹ fail 0` ; puis les trois autres.

ℹ Autonome

Vous allez bloquer : c'est voulu, c'est le métier. Méthode : 1. lisez le message d'erreur en entier ; 2. cherchez-le dans la fiche « Lire un message d'erreur » ; 3. lisez la documentation ; 4. après 15 minutes bloqué sur le même point, demandez.

ℹ Aides

- **Le cours** : le visualiseur pas à pas.
- **L'IA** : CS50.ai (palier IA 1) : demandez ce qu'est une *closure*, avec un autre exemple que celui de l'exercice.
- **MDN** : Closures, en français.

Si l’on bloque

1. Lisez le message d’erreur en entier.
2. Cherchez-le dans la fiche « Lire un message d’erreur ».
3. Relisez le pas à pas ; copiez le dossier de rattrapage de l’étape.
4. Demandez une explication (pas le code) à l’IA du palier.
5. Demandez au formateur ; sur un créneau sans formateur, à un camarade, puis notez la question pour le formateur.

## Sources

MDN Web Docs. s. d. « Closures - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Closures.